

U5-AT1 PYTHON CODE

1. PYTHON CODE:

```
from pyspark import SparkConf, SparkContext

from math import sqrt

# -----
# Hybrid AI Recommendation Engine using PySpark RDD
# -----

conf =
SparkConf().setAppName("HybridRecommendationEngine").setMaster("local[*]")

sc = SparkContext(conf=conf)

# -----
# Sample User-Item Rating Data
# Format: (User, Item, Rating)
# -----

ratings_data = [
    ("U1", "I1", 5),
    ("U1", "I2", 4),
    ("U1", "I3", 2),
    ("U2", "I1", 4),
    ("U2", "I2", 5),
    ("U2", "I3", 1),
    ("U3", "I1", 1),
    ("U3", "I2", 2),
    ("U3", "I3", 5),
```

```
    ("U3", "I4", 4),
    ("U4", "I2", 3),
    ("U4", "I3", 4),
    ("U4", "I4", 5)
]
```

```
ratings = sc.parallelize(ratings_data)
```

```
# -----
# Item Content Information
# Genre / Feature representation
# -----
```

```
items = {
    "I1": {"action": 1, "comedy": 0, "science": 1},
    "I2": {"action": 1, "comedy": 1, "science": 0},
    "I3": {"action": 0, "comedy": 1, "science": 1},
    "I4": {"action": 0, "comedy": 0, "science": 1}
}
```

```
# -----
# 1. Calculate average rating for each item
# -----
```

```
item_ratings = ratings.map(
    lambda x: (x[1], (x[2], 1))
)
```

```
item_average = item_ratings.reduceByKey(
```

```

        lambda a, b: (a[0] + b[0], a[1] + b[1])
    ).mapValues(
        lambda x: x[0] / x[1]
    )

print("\nAverage Item Ratings:")
for item, rating in item_average.collect():
    print(item, round(rating, 2))

# -----
# 2. Collaborative Filtering
# -----

user_ratings = ratings.groupBy(lambda x: x[0]).mapValues(
    lambda x: {item: rating for _, item, rating in x}
)

target_user = "U1"

target_ratings = dict(
    user_ratings.filter(lambda x: x[0] == target_user).collect()
)[target_user]

# Calculate average rating of each item
collaborative_scores = {}

for item in items:
    if item not in target_ratings:
        scores = ratings.filter(

```

```

        lambda x: x[1] == item
    ).map(
        lambda x: x[2]
    ).collect()

if scores:
    collaborative_scores[item] = sum(scores) / len(scores)

# -----
# 3. Content-Based Filtering
# -----

user_profile = {
    "action": 0,
    "comedy": 0,
    "science": 0
}

# Build user profile from previously rated items
for item, rating in target_ratings.items():

    for feature, value in items[item].items():
        user_profile[feature] += value * rating

# Normalize profile
total = sum(user_profile.values())

if total > 0:
    for feature in user_profile:

```

```

        user_profile[feature] /= total

content_scores = {}

for item, features in items.items():

    if item not in target_ratings:

        score = 0

        for feature in features:
            score += user_profile[feature] * features[feature]

        content_scores[item] = score

# -----
# 4. Hybrid Recommendation
# -----

hybrid_scores = {}

for item in content_scores:

    content_score = content_scores.get(item, 0)
    collaborative_score = collaborative_scores.get(item, 0)

    # 60% collaborative + 40% content
    hybrid_score = (
        0.6 * collaborative_score +

```

```

        0.4 * content_score
    )

    hybrid_scores[item] = hybrid_score

recommendations = sorted(
    hybrid_scores.items(),
    key=lambda x: x[1],
    reverse=True
)

print("\nRecommendations for", target_user)

for item, score in recommendations:
    print(item, "Score:", round(score, 3))

# -----
# Intelligent Agent
# -----

class RecommendationAgent:

    def update_feedback(self, item, feedback):

        if feedback == "like":
            print("Agent: Positive feedback received for", item)

        elif feedback == "dislike":
            print("Agent: Negative feedback received for", item)

```

```

else:

    print("Agent: Unknown feedback")

agent = RecommendationAgent()

agent.update_feedback("I4", "like")

sc.stop()

```

2. PYTHON CODE:

```

from pyspark import SparkConf, SparkContext

# -----
# Smart Disaster Detection System
# -----

conf = SparkConf().setAppName("SmartDisasterDetection").setMaster("local[*]")
sc = SparkContext(conf=conf)

# -----
# Multi-source Data
# -----

sensor_data = [
    ("S1", "Flood", 85),
    ("S2", "Flood", 92),
    ("S3", "Flood", 88),
    ("S4", "Earthquake", 20),

```

```
    ("S5", "Earthquake", 75),  
    ("S6", "Earthquake", 90)  
]
```

```
satellite_data = [  
    ("A1", "Flood", 90),  
    ("A2", "Flood", 85),  
    ("A3", "Earthquake", 30),  
    ("A4", "Earthquake", 82)  
]
```

```
social_media_data = [  
    ("P1", "Flood", 80),  
    ("P2", "Flood", 75),  
    ("P3", "Earthquake", 70),  
    ("P4", "Earthquake", 85)  
]
```

```
sensor_rdd = sc.parallelize(sensor_data)  
satellite_rdd = sc.parallelize(satellite_data)  
social_rdd = sc.parallelize(social_media_data)
```

```
# -----  
# Process Sensor Data  
# -----
```

```
sensor_scores = sensor_rdd.map(  
    lambda x: (x[1], x[2])  
).groupByKey().mapValues(  

```



```

        lambda values: sum(values) / len(values)
    )

# -----
# Process Satellite Data
# -----

satellite_scores = satellite_rdd.map(
    lambda x: (x[1], x[2])
).groupByKey().mapValues(
    lambda values: sum(values) / len(values)
)

# -----
# Process Social Media Data
# -----

social_scores = social_rdd.map(
    lambda x: (x[1], x[2])
).groupByKey().mapValues(
    lambda values: sum(values) / len(values)
)

print("\nSensor Scores:")
print(sensor_scores.collect())

print("\nSatellite Scores:")
print(satellite_scores.collect())

```

```

print("\nSocial Media Scores:")
print(social_scores.collect())

# -----
# Data Fusion
# -----

sensor_dict = dict(sensor_scores.collect())
satellite_dict = dict(satellite_scores.collect())
social_dict = dict(social_scores.collect())

disasters = set(sensor_dict.keys()) | \
             set(satellite_dict.keys()) | \
             set(social_dict.keys())

final_scores = {}

for disaster in disasters:

    sensor = sensor_dict.get(disaster, 0)
    satellite = satellite_dict.get(disaster, 0)
    social = social_dict.get(disaster, 0)

    # Weighted data fusion
    final_score = (
        0.4 * sensor +
        0.4 * satellite +
        0.2 * social
    )

```

```
    final_scores[disaster] = final_score

# -----
# Disaster Classification
# -----

print("\nFinal Disaster Detection Results:")

for disaster, score in final_scores.items():

    if score >= 75:

        status = "HIGH RISK"

    elif score >= 50:

        status = "MEDIUM RISK"

    else:

        status = "LOW RISK"

    print(
        disaster,
        "Score:",
        round(score, 2),
        "Status:",
        status
    )

# -----
```

```
# Intelligent Agents
```

```
# -----
```

```
class SensorAgent:
```

```
    def monitor(self):
```

```
        print("Sensor Agent: Monitoring environmental sensors...")
```

```
class SatelliteAgent:
```

```
    def analyze(self):
```

```
        print("Satellite Agent: Analyzing satellite information...")
```

```
class WarningAgent:
```

```
    def issue_warning(self, disaster, score):
```

```
        if score >= 75:
```

```
            print(
```

```
                "WARNING:",
```

```
                disaster,
```

```
                "detected with high risk!"
```

```
            )
```

```
sensor_agent = SensorAgent()
```

```
satellite_agent = SatelliteAgent()
```

```

warning_agent = WarningAgent()

sensor_agent.monitor()

satellite_agent.analyze()

for disaster, score in final_scores.items():
    warning_agent.issue_warning(disaster, score)

sc.stop()

```

3. **PYTHON CODE:**

```

from pyspark import SparkConf, SparkContext

# -----
# AI-Based Fraud Detection using PySpark RDD
# -----

conf = SparkConf().setAppName("FraudDetection").setMaster("local[*]")
sc = SparkContext(conf=conf)

# -----
# Transaction Data
# Format:
# Transaction ID, User, Amount, Location, Hour
# -----

transactions_data = [
    ("T1", "U1", 500, "Chennai", 10),
    ("T2", "U1", 700, "Chennai", 11),

```

```
("T3", "U2", 1200, "Bangalore", 14),  
("T4", "U2", 1500, "Bangalore", 15),  
("T5", "U3", 200, "Chennai", 9),  
("T6", "U3", 250, "Chennai", 10),  
("T7", "U4", 15000, "Mumbai", 2),  
("T8", "U4", 18000, "Delhi", 3),  
("T9", "U5", 300, "Chennai", 12)  
]
```

```
transactions = sc.parallelize(transactions_data)
```

```
# -----  
# Calculate average transaction amount per user  
# -----
```

```
user_amounts = transactions.map(  
    lambda x: (x[1], (x[2], 1))  
)
```

```
user_statistics = user_amounts.reduceByKey(  
    lambda a, b: (  
        a[0] + b[0],  
        a[1] + b[1]  
    )  
)
```

```
user_average = user_statistics.mapValues(  
    lambda x: x[0] / x[1]  
)
```

```

print("\nAverage Transaction Amount:")
for user, amount in user_average.collect():
    print(user, round(amount, 2))

# -----
# Detect anomalies
# -----

average_dict = dict(user_average.collect())

def detect_fraud(transaction):

    transaction_id, user, amount, location, hour = transaction

    average = average_dict[user]

    fraud_score = 0

    # Large transaction
    if amount > average * 5:
        fraud_score += 50

    # Unusual transaction time
    if hour < 5:
        fraud_score += 25

    # Very large amount
    if amount > 10000:

```

```
    fraud_score += 30
```

```
    return (  
        transaction_id,  
        user,  
        amount,  
        fraud_score  
    )
```

```
fraud_results = transactions.map(detect_fraud)
```

```
# -----
```

```
# Classify transactions
```

```
# -----
```

```
def classify(transaction):
```

```
    transaction_id, user, amount, score = transaction
```

```
    if score >= 50:
```

```
        status = "SUSPICIOUS"
```

```
    else:
```

```
        status = "NORMAL"
```

```
    return (  
        transaction_id,
```

```
        user,
```

```
        status
```



```
        amount,  
        score,  
        status  
    )
```

```
classified = fraud_results.map(classify)
```

```
print("\nFraud Detection Results:")
```

```
for result in classified.collect():
```

```
    print(result)
```

```
# -----
```

```
# Intelligent Fraud Detection Agent
```

```
# -----
```

```
class FraudAgent:
```

```
    def analyze(self, transaction):
```

```
        transaction_id, user, amount, score, status = transaction
```

```
        if status == "SUSPICIOUS":
```

```
            print(  
                "Fraud Agent:",  
                "Alert triggered for",  
                transaction_id
```

```

    )

else:

    print(
        "Fraud Agent:",
        transaction_id,
        "is normal."
    )

agent = FraudAgent()

for transaction in classified.collect():
    agent.analyze(transaction)

sc.stop()

```

4. PYTHON CODE:

```

from pyspark import SparkConf, SparkContext

# -----
# Autonomous Supply Chain Optimization
# -----

conf = SparkConf().setAppName("SupplyChainOptimization").setMaster("local[*]")
sc = SparkContext(conf=conf)

# -----

```

```

# Product Demand Data

# Product, Location, Demand
# -----

demand_data = [
    ("Laptop", "Chennai", 100),
    ("Laptop", "Bangalore", 150),
    ("Mobile", "Chennai", 300),
    ("Mobile", "Bangalore", 250),
    ("Tablet", "Chennai", 120),
    ("Tablet", "Bangalore", 100)
]

demand_rdd = sc.parallelize(demand_data)

# -----

# Calculate total demand
# -----

total_demand = demand_rdd.map(
    lambda x: (x[0], x[2])
).reduceByKey(
    lambda a, b: a + b
)

print("\nTotal Demand:")

for product, demand in total_demand.collect():
    print(product, demand)

```

```

# -----
# Calculate average demand by location
# -----

location_demand = demand_rdd.map(
    lambda x: (x[1], (x[2], 1))
).reduceByKey(
    lambda a, b: (
        a[0] + b[0],
        a[1] + b[1]
    )
)

average_location_demand = location_demand.mapValues(
    lambda x: x[0] / x[1]
)

print("\nAverage Demand by Location:")

for location, demand in average_location_demand.collect():
    print(location, round(demand, 2))

# -----
# Inventory Data
# -----

inventory_data = [
    ("Laptop", 120),

```

```
    ("Mobile", 400),  
    ("Tablet", 80)  
]
```

```
inventory_rdd = sc.parallelize(inventory_data)
```

```
inventory = dict(inventory_rdd.collect())
```

```
# -----  
# Determine inventory requirement  
# -----
```

```
total_demand_dict = dict(total_demand.collect())
```

```
print("\nInventory Optimization:")
```

```
for product, demand in total_demand_dict.items():
```

```
    current_inventory = inventory.get(product, 0)
```

```
    required_inventory = demand * 1.2
```

```
    if current_inventory < required_inventory:
```

```
        additional = required_inventory - current_inventory
```

```
    print(  
        product,  
        "-> Reorder",
```

```

        round(additional),
        "units"
    )

else:

    print(
        product,
        "-> Inventory sufficient"
    )

# -----
# Transportation Optimization
# -----

transport_data = [
    ("Chennai", "Bangalore", 350, 5000),
    ("Chennai", "Hyderabad", 630, 7000),
    ("Bangalore", "Hyderabad", 570, 6000)
]

transport_rdd = sc.parallelize(transport_data)

# Calculate cost per kilometer
transport_cost = transport_rdd.map(
    lambda x: (
        x[0],
        x[1],
        x[3] / x[2]
    )
)

```

```
)  
)
```

```
print("\nTransportation Cost per KM:")
```

```
for source, destination, cost in transport_cost.collect():
```

```
    print(  
        source,  
        "->",  
        destination,  
        "Cost/KM:",  
        round(cost, 2)  
    )
```

```
# -----  
# Intelligent Agents  
# -----
```

```
class InventoryAgent:
```

```
    def make_decision(self, product, demand, inventory):
```

```
        if inventory < demand:
```

```
            print(  
                "Inventory Agent:",  
                "Reorder required for",  
                product
```

```
)
```

```
else:
```

```
    print(
```

```
        "Inventory Agent:",
```

```
        "Stock sufficient for",
```

```
        product
```

```
    )
```

```
class LogisticsAgent:
```

```
    def optimize(self):
```

```
        print(
```

```
            "Logistics Agent:",
```

```
            "Selecting cost-efficient transportation routes."
```

```
        )
```

```
class DemandForecastAgent:
```

```
    def forecast(self):
```

```
        print(
```

```
            "Demand Forecast Agent:",
```

```
            "Analyzing historical demand."
```

```
        )
```



```

inventory_agent = InventoryAgent()
logistics_agent = LogisticsAgent()
forecast_agent = DemandForecastAgent()

forecast_agent.forecast()
logistics_agent.optimize()

for product, demand in total_demand_dict.items():

    inventory_agent.make_decision(
        product,
        demand,
        inventory.get(product, 0)
    )

sc.stop()

```

5. **PYTHON CODE:**

```

from pyspark import SparkConf, SparkContext

# -----
# Personalized Learning Analytics using PySpark
# -----

conf = SparkConf().setAppName("PersonalizedLearning").setMaster("local[*]")
sc = SparkContext(conf=conf)

```

```
# -----  
# Student Performance Data  
# Student, Subject, Score, Study Hours  
# -----
```

```
student_data = [  
    ("S1", "Python", 85, 5),  
    ("S1", "AI", 90, 6),  
    ("S1", "Math", 80, 4),  
  
    ("S2", "Python", 55, 2),  
    ("S2", "AI", 60, 3),  
    ("S2", "Math", 50, 2),  
  
    ("S3", "Python", 75, 4),  
    ("S3", "AI", 70, 4),  
    ("S3", "Math", 65, 3),  
  
    ("S4", "Python", 40, 1),  
    ("S4", "AI", 45, 2),  
    ("S4", "Math", 50, 2)
```

```
]
```

```
students = sc.parallelize(student_data)
```

```
# -----  
# Calculate average score for each student  
# -----
```

```

student_scores = students.map(
    lambda x: (x[0], (x[2], 1))
)

student_statistics = student_scores.reduceByKey(
    lambda a, b: (
        a[0] + b[0],
        a[1] + b[1]
    )
)

average_scores = student_statistics.mapValues(
    lambda x: x[0] / x[1]
)

print("\nAverage Student Scores:")

for student, score in average_scores.collect():

    print(
        student,
        "Average:",
        round(score, 2)
    )

# -----
# Calculate average study hours
# -----

```

```

study_hours = students.map(
    lambda x: (x[0], (x[3], 1))
)

study_statistics = study_hours.reduceByKey(
    lambda a, b: (
        a[0] + b[0],
        a[1] + b[1]
    )
)

average_hours = study_statistics.mapValues(
    lambda x: x[0] / x[1]
)

print("\nAverage Study Hours:")

for student, hours in average_hours.collect():

    print(
        student,
        "Hours:",
        round(hours, 2)
    )

# -----
# Identify learning level
# -----

```

```
score_dict = dict(average_scores.collect())

def classify_student(data):

    student, score = data

    if score >= 80:

        level = "Advanced"

    elif score >= 60:

        level = "Intermediate"

    else:

        level = "Beginner"

    return student, score, level

student_levels = average_scores.map(classify_student)

print("\nStudent Learning Levels:")

for result in student_levels.collect():

    print(result)

# -----
# Generate Personalized Recommendations
# -----
```

```
def recommend(data):

    student, score, level = data

    if level == "Advanced":

        recommendation = (
            "Provide advanced projects and challenging content"
        )

    elif level == "Intermediate":

        recommendation = (
            "Provide practice exercises and intermediate content"
        )

    else:

        recommendation = (
            "Provide basic concepts, tutorials and additional practice"
        )

    return (
        student,
        level,
        recommendation
    )
```

```
recommendations = student_levels.map(recommend)
```

```
print("\nPersonalized Recommendations:")
```

```
for recommendation in recommendations.collect():
```

```
    print(recommendation)
```

```
# -----
```

```
# Intelligent Learning Agent
```

```
# -----
```

```
class LearningAgent:
```

```
    def analyze(self, student, level):
```

```
        print(
            "Learning Agent:",
            "Analyzing learning behavior of",
            student
        )
```

```
        if level == "Beginner":
```

```
            print(
                "Action:",
                "Recommend basic learning materials."
            )
```

```
elif level == "Intermediate":
```

```
    print(  
        "Action:",  
        "Recommend practice exercises."  
    )
```

```
else:
```

```
    print(  
        "Action:",  
        "Recommend advanced learning projects."  
    )
```

```
agent = LearningAgent()
```

```
for student, score, level in student_levels.collect():
```

```
    agent.analyze(student, level)
```

```
sc.stop()
```